

Embedded system Concept

* ① Computing system Definition

- ↳ device → some operation (multitask)
- ↳ Computing system Component (
 - ↳ Processor → Multi Core
 - ↳ Memory → RAM :: by Giga
ROM :: by Tera
 - ↳ I/O Peripherals

② Embedded system

- ↳ it's Computing system with limit Resource
 - Processor
 - Single Core
 - Double Core
 - Memory
 - ROM 32KB
 - RAM 2KB
 - I/O limited
- ↳ to do specific task

③ Embedding system

- ↳ it's Multi Embedded systems communicate together

* Compare Soc Vs SoC

Factors	SOC	SoC
Performance	—	—
Size	Memory :: — volume :: Low	Memory :: — volume :: High
Cost	Low	High
Power Consumption	Low	High
Configurability	N/A Production after Design	Easy For developer & Design Phase

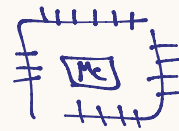
* Mc → Micro Controller → Soc

- Process
- Memory
- I/O

* ECU → Electronic Control Unit

- Electronic Component

* Development kit



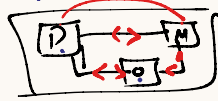
* Embedded Challenges

- Performance → Target High
- Cost → Low
- Size (Memory/Volume) → Low
- Power Consumption → Low

* How design Embedded system?

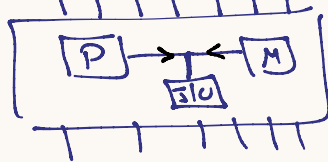
↳ ① system on Board (SoB)

- ↳ White Board / PCB
- ↳ Bus → Wire / track
- ↳ Use when H.W development



↳ ② system on chip (SoC) (Mc)

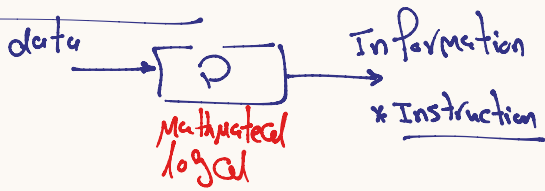
- ↳ inside chip
- ↳ Bus :: tunnel
- ↳ Use when Production Phase



* Embedded system → Processor → ①

- Memory
- I/O Peripherals

1) Processor



* Notes about Processor

① Processor need Power & Clock to work

② Processor work with Memory only

③ clock it's square wave



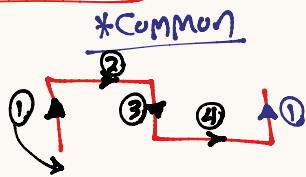
④ Processor → Machine cycle

① Fetch

② de Code

③ execute

④ Interrupt Req



⑤ Machine cycle must be to execute Instruction

* Processor History

→ ① Processor → vacuum tube

→ ② Micro Processor → transistor

→ [today] : Processor / Micro Processor

→ ③ CPU: Central Processor unit

→ in Embedded system.

Single Core

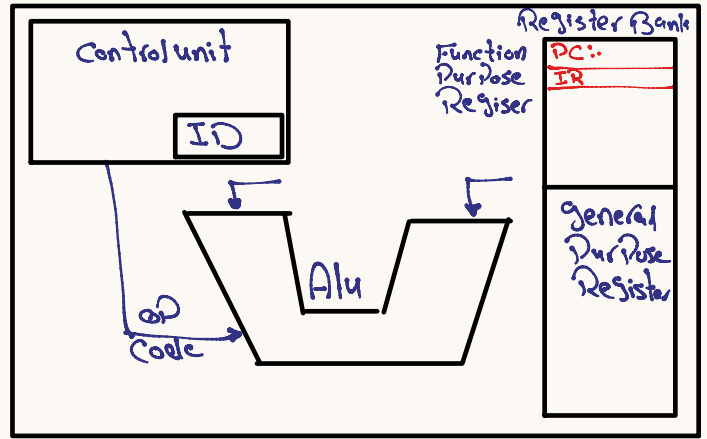
Double Core

MP = CPU = Processor

CPU

GPU

* Inside Processor



* Machine cycle Sequence

① Fetch : Processor get the Instruction will be executed from Memory (ROM) → (Code Memory)

→ ① Control unit dedicate the Fetch time

→ ② Control unit Read PC Register to get the Address of Instruction.
* PC: Program Counter store the Instruction Address will be executed.

→ ③ Processor will be Communicate with Address to get Instruction.

→ ④ Control unit store the Instruction in the IR Register.

* IR: Instruction Register → Store Instruction

→ ⑤ Control unit update the PC to next Instruction Address (1)

② de Code : Processor understand Instruction to get

→ Control unit
Instruction
ID Decoder

→ ① op Code [operation]

→ ② operand

→ ③ who will be execute.

For every Processor have Instruction set

Add

Sub

or

011

101

001

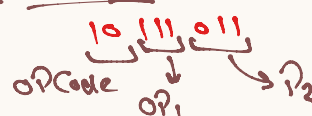
op Code

op₁

op₂

unique

* Instruction Format



* op Code → who will be execute

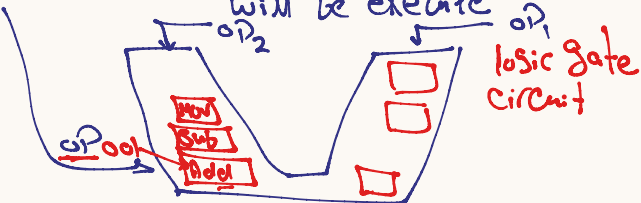
Add

Alu

Store

Control unit

③ execute : Control unit will be select if his execute Instruction or the ALU → Arithmetic Logical unit will be execute



ISA :: Instruction Architecture

* RISC
 → Reduce Instruction set Architecture
 → range ≈ 100 Instructions
 → Almost Instruction will be executed in 1 clk

① Performance (Risc)
 ↳ 1 clk
 * $5 + 4 \rightarrow 1 \text{ clk}$
 * $10 \times 100 \rightarrow \text{Not support}$
 ↳ $100 + 100 + 100 + 100 + 100$
 ↳ 10 clk cycle

the same

② Cost (Risc)
 H.W = Simple
 Low Cost
 S.W = Complex
 High Cost

the same

③ Size (Risc)
 ↳ Alu → small Instruction set
 ↳ small size

↳ I.D :: Hardwired
 ↳ H.W circuit
 ↳ very fast
 ↳ 1 clk

the same

④ Power Consumption (Risc)
 ↳ work 1 circuit in Alu

the same

ex: Intel → Cisc
 ARM → Risc

Apple → change Processor From Intel into ARM

↳ low Power
 ↳ 3.3V

* CISC
 * Complex Instruction set Architecture.
 * range ≈ 1000 Instructions
 * Almost Instruction will be executed in 4 clk cycle

① Performance (Cisc)
 ↳ 4 clk
 * ex $5 + 4 \rightarrow 4 \text{ clk}$
 * ex $10 \times 100 \rightarrow 4 \text{ clk}$

the same

② Cost (Cisc)
 H.W = Complex
 High Cost
 S.W = Simple
 Low Cost

the same

③ Size (Cisc)
 ↳ Alu → large Instruction set
 ↳ large size

↳ I.D :: Microprogrammed
 ↳ small chip
 ↳ S.W algorithm search about op Code
 ↳ small
 ↳ 4 clk

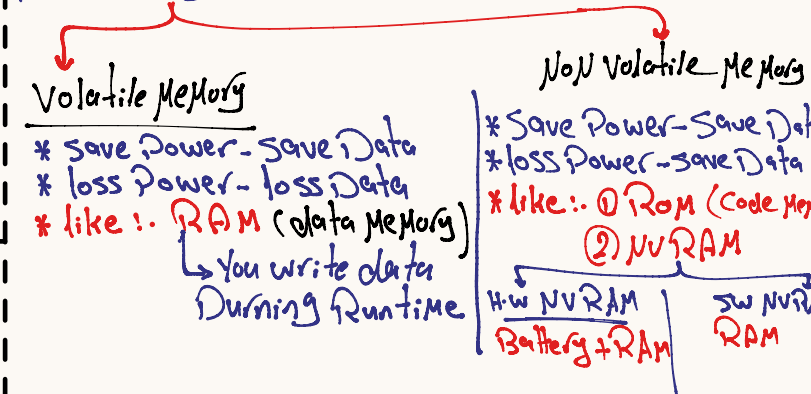
the same

④ Power Consumption (Cisc)
 ↳ works 1 circuit in Alu

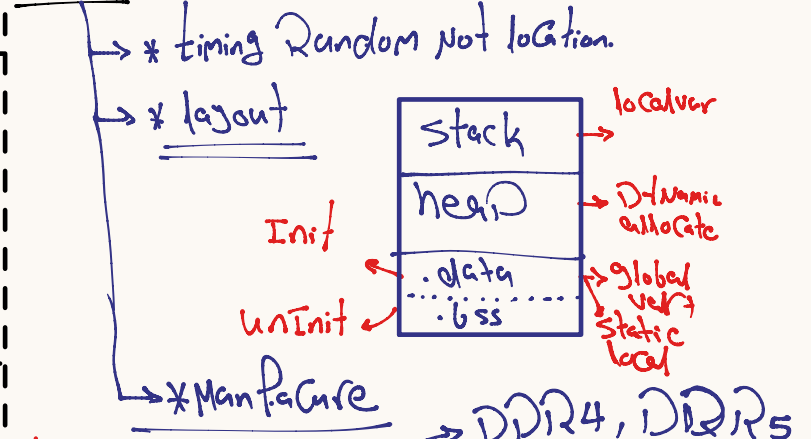
the same

Note in side Processor
 FPR :: Function Purpose Register
 ↳ PC → Program Counter
 ↳ IR → Instruction Reg
 ↳ ACC → Accumulator
 ↳ PSW → Processor status word
 ↳ SP → stack Pointer

② Memory



RAM :: Random Access Memory



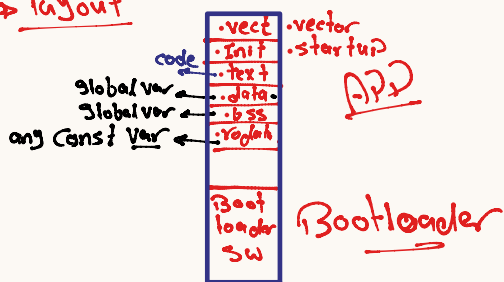
↳ ① Based on Cap :: Dynamic RAM
 ↳ low cost ↳ large scale
 ↳ After time Cap discharge
 ↳ Solve → Periodic charge

↳ ② Based on transistor (Bjt)
 ↳ static RAM :: SRAM
 ↳ High Cost
 ↳ save value
 ↳ used in Embedded

in Atmega32 → 2KB
 ↳ SRAM

ROM: Read Only Memory.

- Processor Can Read From this Memory
- Code Memory
- layout



- Manufacture: Based on transistor (MOSFET)
- to write on ROM need High Power
- Need to Programmable kit to upload code on ROM (USB asb)

ROM type

1) Masked ROM → *

- factory will be upload code.
- You can't change code

2) OTP → one time Programmable (ROM)

- Memory is empty → from factory
- You can upload your code one time only.

3) EPROM → Erasable Programmable (ROM)

- You can upload your code multi time
- You can erase ROM by (UV) → long time

4) EEPROM → Electricity Erasable Programmable ROM

- You can upload your code multi time
 - You can erase ROM by electricity.
 - write bit by bit
 - Can write data and modify it during runtime
- Code
data
save data

5) Flash ROM

- Erase by electricity
- upload code multi time
- word by word (Page / Page)
- word / Page → No of byte.
- save code only.

Memory Conclusion

RAM: Runtime (data)

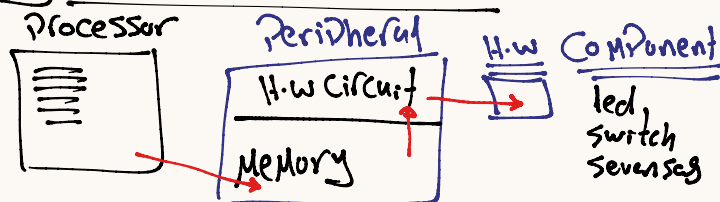
ROM: Code

in Embedded system.

* Code → Flash

* data → SRAM
→ EEPROM

3) I/O Peripheral

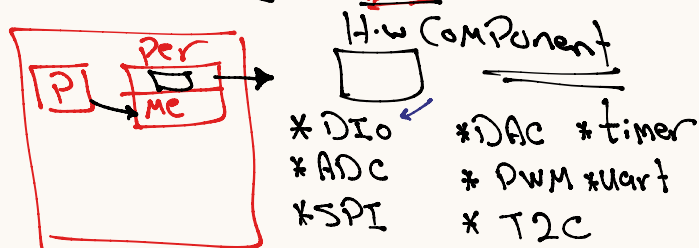


* it's gateway between Processor & H.W Component

* For each H.W Component specific peripherals

Bluetooth → UART → Processor
led → DIO → Processor

* Peripheral → inside MC



For each Peripheral have H.W Circuit & Peripheral Memory